

Secure Data Lakes

Confidentiality & Integrity with
Trusted Execution Environments

Index

01 Introduction & Motivation

02 Data Lakes Context

03 Intel SGX & Remote Attestation

04 State of the Art

05 System Architecture

06 Implementation Phases

07 Security Analysis

08 Conclusion

The Challenge

Why do data lakes need stronger security guarantees?

INTRODUCTION

Scale

Massive repositories designed for multi-format, multi-schema storage across healthcare, finance, and research.

Openness

Collaborative architecture is both the main advantage and the main vulnerability (multiple sources, multiple users).

Regulation

Data breaches, even partial, trigger severe violations under GDPR and HIPAA, with legal consequences for data subjects.

Data lakes aggregate sensitive data from many sources and expose it to many users, multiplying the attack surface.

The Medical Domain

MOTIVATION

Potential

- Early detection of epidemic outbreaks
- Drug efficacy evaluation at population scale
- Collaboration between hospitals, research institutes, and public health authorities
- Acceleration of scientific discovery

Risk

The same data enabling these analyses are sensitive personal records:

- Diagnoses
- Lab results
- Treatment history

Partial exposure can have serious consequences for data subjects and constitutes regulatory violations under GDPR & HIPAA.

Traditional Protections Are Insufficient

CORE PROBLEM

✓ Encryption at rest

✓ Role-based access control

✓ Access auditing

BUT

The Computation Problem

To execute a query, data must be decrypted, exposing it in memory to the OS, hypervisor, and cloud operator. A privileged adversary can bypass all logical controls and read process memory directly.

⚠ Cloud Operator

⚠ Hypervisor

⚠ Operating System

⚠ Other Processes

All have potential access to plaintext data during computation

Data Lakes

Centralized repositories for raw, heterogeneous data

CONTEXT

A Data Lake stores massive volumes of structured, semi-structured, and unstructured data in its native (raw) format. Unlike traditional databases, it adopts a schema-on-read approach, a no rigid structure required at write time.

Schema-on-read

Structure defined at query time, not at ingestion. Maximizes flexibility.

Heterogeneous sources

Ingests from legacy systems, IoT, APIs, data brokers simultaneously.

Collaborative

Researchers explore raw data freely without affecting operational sources.

Big Data stack

Built on Apache Hadoop/HDFS, Spark, or cloud object storage at scale.

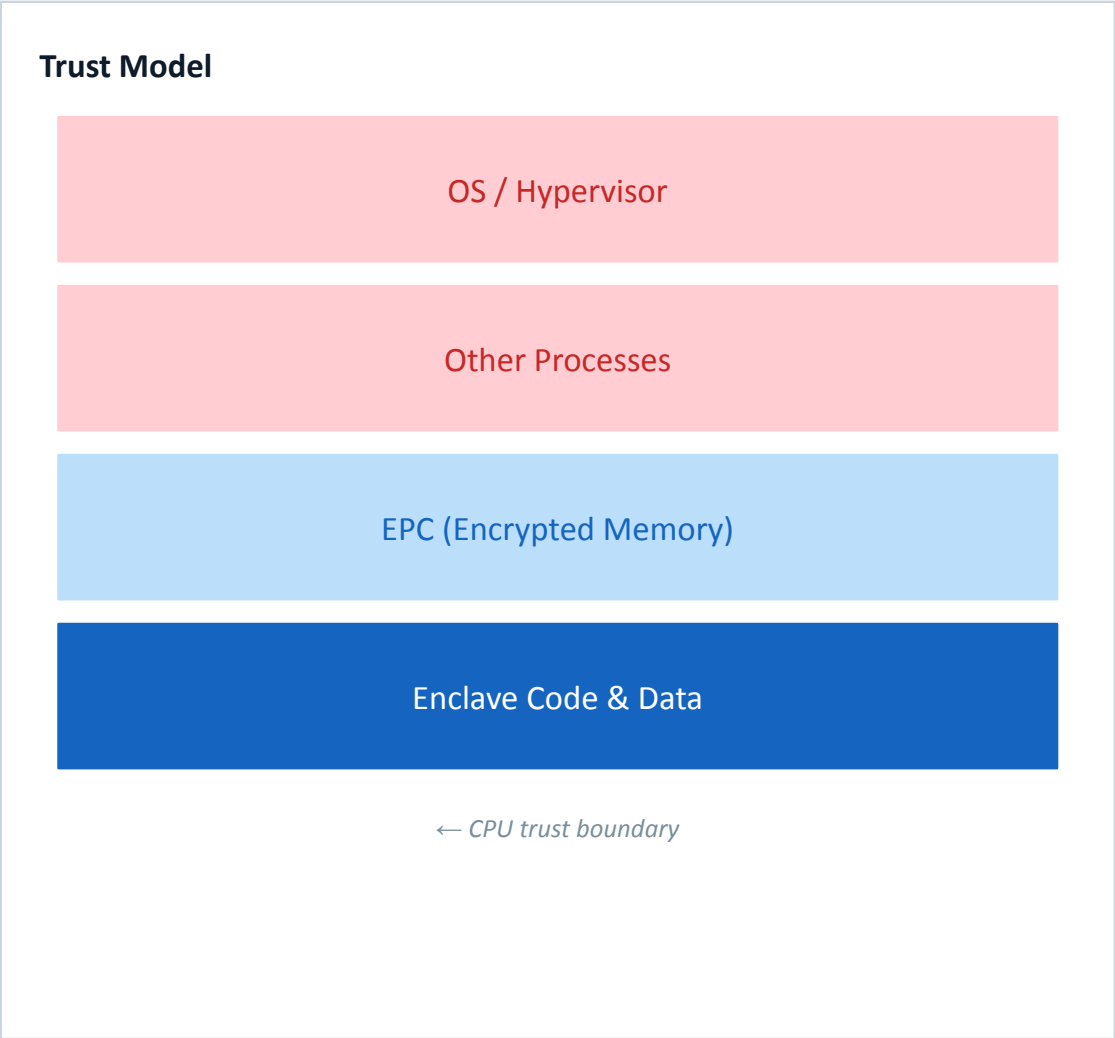
Intel SGX - Software Guard Extensions

TECHNICAL FOUNDATIONS

Core Idea

SGX is an x86 ISA extension (since Skylake 2015) that creates isolated memory regions called enclaves. Trust is placed exclusively in the Intel CPU. All other components (OS, hypervisor, BIOS, other processes) are considered potentially malicious.

EPC	Enclave Page Cache - hardware-encrypted physical memory (AES-128 CTR + CBC-MAC)
Keys	Generated by CPU at boot, never leave the chip
Isolation	Any EPC access outside enclave context yields only ciphertext
Lifecycle	ECREATE → EADD → EEXTEND → EINIT → sealed with MRENCLAVE



Remote Attestation & DCAP

TECHNICAL FOUNDATIONS

Remote attestation lets an external party verify it is communicating with a legitimate, unmodified enclave running on a genuine Intel processor. Without it, an adversary could impersonate the enclave to obtain encryption keys.

EPID (Classic)

Depends on Intel central servers for signature verification (online dependency).

DCAP (Since 2019)

ECDSA attestation verification runs locally in the datacenter, no real-time Intel connectivity required.

DCAP Attestation Flow

- 1 Enclave generates SGX Report (MRENCLAVE + custom data, e.g. ephemeral public key)
- 2 Quoting Enclave signs Report with ECDSA key derived from CPU Provisioning Cert Key (PCK)
- 3 Verifier uses local PCCS to fetch PCK certs, TCB Info, QE Identity
- 4 After success: verifier cryptographically confirms enclave code integrity and CPU TCB level

Applied Cryptography Inside the Enclave

TECHNICAL FOUNDATIONS

Key Exchange ECDH on Curve25519	Secure channel between participant and enclave. Enclave public key certified during attestation.
Data Encryption AES-256-GCM	Each record encrypted by participants before submission. Provides both confidentiality and authenticity.
Result Auth. HMAC-SHA256	Query results signed with a key derived from the shared session secret.
Key Derivation HKDF + X25519	Symmetric session key (AES-256) derived from ECDH output via HKDF.

State of the Art

Related work on confidential analytics with SGX

LITERATURE

Opaque

NSDI 2017

Apache Spark + SGX for SQL queries over distributed encrypted data. Introduces oblivious mode to hide memory access patterns.

EnclaveDB

IEEE S&P 2018

Full relational database (Hekaton engine) running entirely inside SGX. Guarantees confidentiality, integrity, and anti-rollback.

VC3

IEEE S&P 2015

Confidential MapReduce jobs in the cloud. Users verify code ran correctly on correct data without exposing either to the cloud operator.

DuckDB-SGX2

arXiv 2024

OLAP engine (DuckDB) inside SGX2 via Gramine LibOS. TPC-H overhead: 2–4× for in-EPC datasets. Validates our approach.

Medical context: EHDEN (federated analysis of clinical data) and SMPC4Health show complementary approaches. TEE + differential privacy is an active research frontier.

TEE Landscape

Why Intel SGX for this project?

LITERATURE

Platform	Ecosystem	Integrity Guarantees	App Migration	Best Use Case
Intel SGX	Mature	Hardware-level (process)	Requires rewrite	Our choice
AMD SEV	Growing	VM-level (SEV-SNP)	Easy (whole VM)	Easier migration, weaker TCB
ARM TrustZone	Widespread	Zone-level	Hard	Mobile / DRM focus
RISC-V Keystone	Research	Configurable	Hard	Research platform only

SGX offers the best balance of ecosystem maturity, security expressiveness, and performance for this context.

System Architecture

Four-component design: data never leaves the enclave in plaintext

ARCHITECTURE

Core principle: plaintext data never leaves the enclave. The cloud operator hosting the server has zero access to data, keys, or unauthenticated results.

01

Consortium Clients
Submit encrypted records. Issue queries. Perform remote attestation before any data sharing.

02

Secure Processing Server
Hosts the SGX enclave. Executes all queries inside the enclave. Untrusted host component.

03

Attestation & Key Mgmt
Mediates trust establishment between clients and enclave via DCAP quote verification.

04

Persistent Storage
Maintains encrypted records on disk. Data sealed with enclave master key, surviving reboots.

Data Ingestion & Query Flow

ARCHITECTURE

Ingestion Protocol

- 1

Client requests Report from enclave (ECDH public key + MRENCLAVE)
- 2

Client verifies Report via DCAP QVL - checks ECDSA chain to Intel root CA
- 3

ECDH key exchange → session key derived via HKDF (AES-256)
- 4

Records serialized (Protobuf), encrypted with AES-256-GCM + random nonce
- 5

Enclave decrypts, verifies GCM tag, stores in sealed memory

Supported SQL Operations

- COUNT - records matching a criterion
- SUM / AVG - numeric field aggregation
- GROUP BY - aggregation by category (diagnosis, age group)
- Pearson correlation - for drug efficacy analysis

Result Security Checks

- Minimum K=5 record threshold (k-anonymity)
- Authorization check per data category
- Result signed with HMAC-SHA256 (session key)

Implementation: 4 Phases

IMPLEMENTATION

Phase 1

Minimal Enclave

Direct SGX SDK. Simulated DCAP attestation with ECDSA-signed quotes (*ecall_generate_report*), per-hospital session key exchange (*ecall_finish_key_exchange*), AES-128-GCM encrypted upload (*ecall_upload_data*), and aggregate queries AVG/MIN/MAX/COUNT with diagnosis filtering (*ecall_run_query*). Runs in simulation mode (SGX_MODE=SIM).

Phase 2

Multi-Contributor

Independent session contexts per participant. Internal keystore with TTL-based expiry. Long-term key derivation from processor sealing key (survives reboots).

Phase 3

DuckDB Query Engine

DuckDB integrated via Gramine LibOS. Full SQL subset for medical analytics. Timing side-channel mitigations: response padding + random latency.

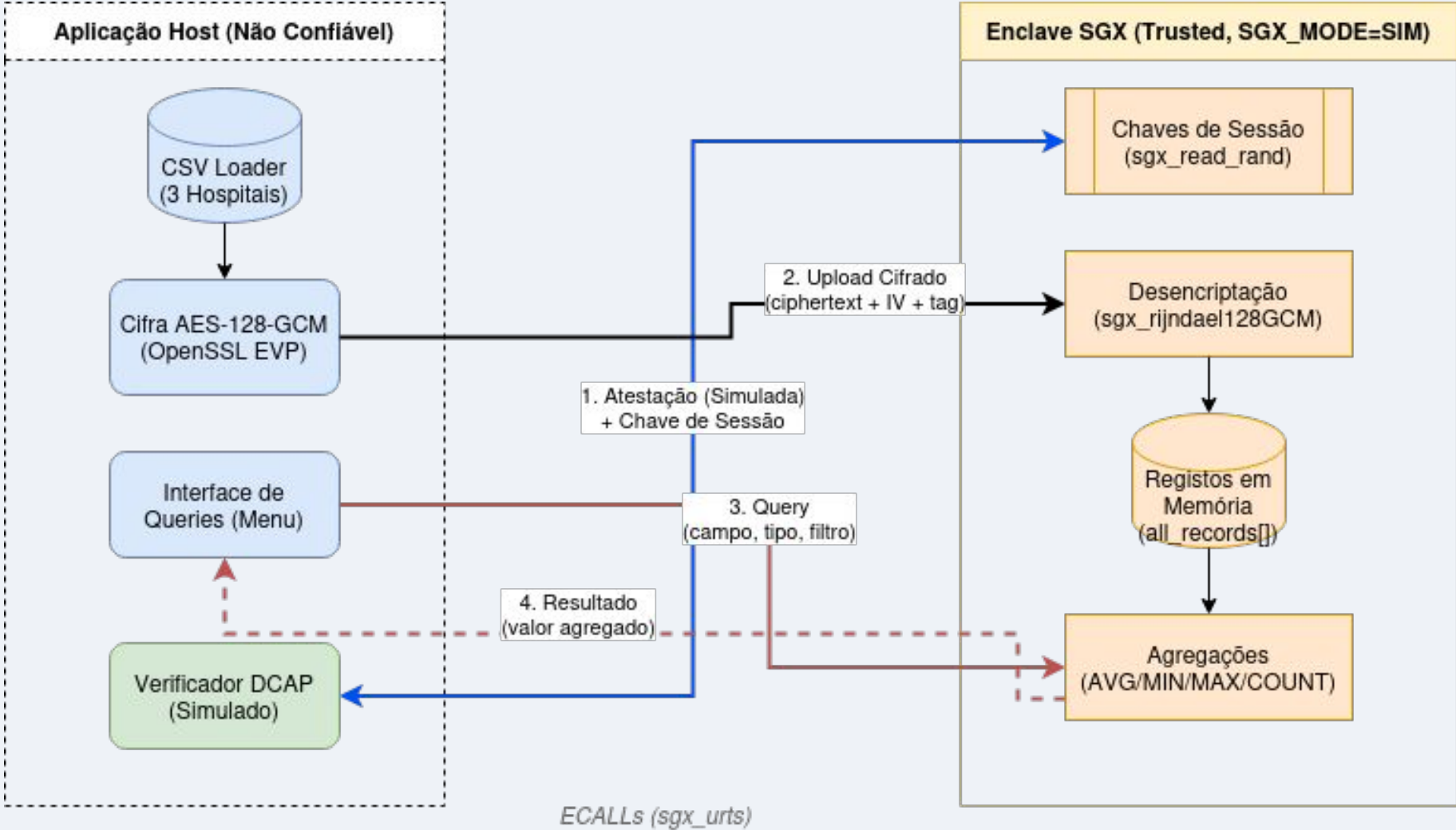
Phase 4

Full DCAP Attestation

Complete DCAP pipeline with libsgx-dcap-ql and libsgx-dcap-quoteverify. Local PCCS for certificate caching. Simulation mode documented for non-SGX hardware.

POC - Architecture Diagram

IMPLEMENTATION



POC - Demo

IMPLEMENTATION

```
) ~/sgx_projects/my_first_enclave
(base) afartur@silvertech:~/sgx_projects/my_first_enclave$ |
```


Security Analysis

Threat model and guarantees

SECURITY

Formal Guarantees

- ✓

Confidentiality
Individual records never revealed outside the enclave; including to the cloud operator
- ✓

Integrity
Query results faithfully reflect current data; tampering is detectable
- ✓

Authenticity
Participants can cryptographically verify correct, unmodified enclave code
- ✓

Freshness (partial)
Monotonic counters protect against simple rollback; full freshness requires TPM

Known Threats & Mitigations

- Cache side-channels**

Flush+Reload, Prime+Probe: mitigated with constant-time algorithms and Spectre V2 patch (-mindirect-branch=thunk)
- Page fault attacks**

OS controls page tables: SGX2 EDMM partially mitigates. In-EPC operation for sensitive data.
- Statistical inference**

k-anonymity (K=5 minimum threshold). Differential privacy (Laplace/Gaussian) is the priority extension.
- EPC size limits**

Pre-Ice Lake: 92 MB usable. Chunk-based processing for large datasets. Ice Lake+: up to 512 GB EPC.

Limitations & Future Work

SECURITY

No Full Obliviousness

Memory access patterns still potentially visible via page faults on older SGX. Oblivious RAM is future work.

Partial Freshness

Current monotonic counters don't fully prevent rollback without hardware TPM support.

k-Anonymity Only

Threshold-based protection doesn't defend against adaptive multi-query adversaries. Differential privacy is the natural next step.

EPC Constraints

Pre-Ice Lake: 92 MB EPC limits dataset size. Chunk-processing workaround adds complexity.

Without DCAP-capable hardware, attestation is tested in simulation mode only. Production deployment requires real SGX hardware.

Conclusion

1

Secure data lake prototype for collaborative medical data analysis across multiple hospitals, using Intel SGX as the core security primitive. Individual patient data never leaves the enclave.

2

End-to-end flow validated: simulated remote attestation with MRENCLAVE verification, per-hospital session keys, AES-128-GCM encrypted upload, in-enclave decryption with GCM tag integrity check, and aggregate queries (AVG, MIN, MAX, COUNT) with diagnosis filtering.

3

The split-trust separation (trusted vs. untrusted code) demonstrates that sensitive data can be processed in an environment where the infrastructure operator has no access to plaintext data

4

Current limitations are deliberate for this first phase: simulation mode (no real memory isolation), unilateral key exchange, no Intel certificate chain verification, and no data persistence across sessions.

5

Next step: migration to real SGX hardware with full DCAP attestation (PCCS), mutual ECDH key exchange, sealing for encrypted persistence, and k-anonymity thresholds on query results.